

Improving Linear Attention’s Performance via Token Gating in Vision Transformer

Chi-Van Vong , Minh-Quan Pham Nguyen 

Faculty of Information Technology, University of Science, Ho Chi Minh City, Viet Nam
Vietnam National University Ho Chi Minh City, Viet Nam

{vcvan2424, pnmquan2414}@apcs.fitus.edu.vn

Abstract

While softmax attention is widely used recently, its quadratic complexity leads to high computational costs. Recent linear attention mechanisms mitigate the quadratic bottleneck but still redundantly process uninformative background tokens, wasting computational budget and limiting the model accuracy. In this study, we propose a gating framework to evaluate the importance of each token. Specifically, we use Multi-Agent Reinforcement Learning (MARL), where each visual token acts as an autonomous agent that utilizes a shared Actor-Critic policy to output continuous survival weights, gating information flow into the recurrent memory. For image classification tasks, our model reached accuracies of 48.88% and 58.37% on the datasets ImageNet100 and CIFAR-10 respectively. While the framework remains in its early stages of development and current performance metrics are modest, the resulting attention maps suggest that this gating approach holds potential for future optimization. Code can be found at [here](#).

1. Introduction

While softmax attention has become a cornerstone of modern deep learning architectures, its $\mathcal{O}(n^2)$ computational complexity poses a significant bottleneck. This limitation is particularly pronounced in Vision Transformers (ViTs), where the number of visual tokens scales quadratically with the input image resolution. To address the issue, many works attempted to group vision tokens [2, 4, 10]. Although this method reduces the complexity, the Transformer loses its ability to perceive information globally. Other works replace softmax attention with **linear attention** which helps reduce the quadratic complexity [1, 5–7]. However, linear attention’s inability to model complex information yields unsatisfactory results due to its low-rank nature and magnitude negligence. Many works have attempted to improve

linear attention by tackling its low-rank nature [5]. Linear attention replaces softmax with kernel feature maps which causes the loss of sharpness. Therefore, it can not be as selective as softmax attention which can place most of its attention weights on a few relevant tokens. To address this, we apply a MARL gating mechanism to assign an importance score for each token, in which each agent will correspond to a visual token. We hypothesize that this will enable the agents to learn an optimal survival policy, ensuring that only the most salient and task relevant features are propagated into the recurrent memory. When trained from scratch and evaluated on the ImageNet100 dataset, the model achieved an accuracy of 48.88%. Additionally, by transferring the trained backbone to CIFAR-10, we obtained an accuracy of 58.37%. Although the proposed framework requires further refinement and the preliminary quantitative results remain modest, the generated saliency maps demonstrate that the gating mechanism successfully identifies important visual features.

2. Related Work

2.1. Vision Transformer

Transformer [18] model was initially created for NLP tasks and achieved remarkable success. However, they did not have much impact on Computer Vision (CV) until Dosovitskiy *et al.* [15] introduced ViT, which splits the image into many small patches and feeds them as tokens to the model. Although this approach demonstrated strong performance in many CV tasks, its quadratic complexity due to self-attention makes it computationally expensive to process high-resolution images. To address this, Microsoft Research proposed Swin Transformer [11], which restricts attention to local windows, inspiring subsequent window-based variants. Other methods reduce attention cost by compressing keys or values using linear projections, strided convolutions, pooling, or patch clustering [1, 7–9, 13, 17]. Additionally, some variants explore alternative designs such as channel attention and softmax-free attention

mechanisms.

2.2. Linear Attention

Linear attention [17] mitigates the quadratic complexity of softmax attention by using kernel functions to approximate the softmax operator, thereby achieving linear complexity. However, linear attention typically underperforms compared to standard softmax attention due to the inherent limitations and information loss associated with this approximation.

FLattenTransformer [7] introduces a focused function to enhance the ability of linear attention to concentrate on relevant information, while MLLA [8] establishes a link between linear attention and the Mamba model [6]. Many other approaches either rely on kernel-based approximations of the softmax function or integrate depthwise convolutions to improve feature diversity. Recently, Fan *et al.* [5] addressed a fundamental limitation of linear attention by identifying the low-rank nature of its feature representations as a major cause of performance degradation. To overcome this, they proposed Rank-Augmented Linear Attention (RALA), which utilizes a softmax-based ranking to restructure token weights inside the memory buffer, thereby enhancing information richness and increasing the matrix rank. However, applying a global softmax over the entire image can cause the overall saliency to become saturated. Motivated by this, we adopt the overall framework from RALA but replace the global softmax ranking with our proposed MARL gating mechanism.

2.3. Token Gating

Vision Transformers suffer from high computational costs proportional to the number of input tokens. Hence, many attempts have been made to reduce redundant tokens while maintaining high performance. Traditional dynamic methods like DynamicViT [15] and TokenLearner [16] adaptively drop or condense tokens. Adaptive pruning techniques such as AdaViT [14] use attention metrics or learned masks to select important tokens. Recent methods introduce the use of Reinforcement Learning (RL) to learn pruning policies. Specifically, Lu *et al.* [12] formulate token pruning as a sequential decision-making problem using MARL, where each visual token is considered an agent. Our method builds upon this MARL framework but produces continuous survival weights rather than relying on discrete keep-or-prune actions.

3. Method

3.1. Preliminary

Softmax Attention. Softmax attention serves as the core mechanism in standard Transformers. Let Q, K, V denote the query, key, and value matrices for a sequence of N

tokens. The parallel training formulation, incorporating a causal mask M where $M_{i,j} = -\infty$ if $j > i$ and 1 otherwise, is defined as $O = \text{softmax}(QK^\top \odot M)V$. During iterative inference, the output for the t -th step is computed as:

$$o_t = \sum_{j=1}^t \frac{\exp(q_t^\top k_j)}{\sum_{l=1}^t \exp(q_t^\top k_l)} v_j \quad (1)$$

While powerful, softmax attention suffers from a quadratic time complexity $\mathcal{O}(N^2)$ with respect to the sequence length, alongside a linear memory growth $\mathcal{O}(N)$ during inference due to the KV-cache. In vision tasks, where the number of image tokens scales quadratically with resolution, these computational bottlenecks become prohibitive for high-resolution ViTs.

Linear Attention and Matrix-Valued Hidden States. Linear attention addresses this quadratic bottleneck by approximating the softmax similarity matrix. It replaces the softmax operation with an element-wise nonlinear feature mapping $\phi(\cdot)$ (e.g., ReLU or ELU) to ensure non-negative similarities, yielding the parallel formulation $O = \phi(Q)\phi(K)^\top V$. By leveraging the associative property of matrix multiplication, the iterative inference can be rewritten to reveal a matrix-valued hidden state. The unnormalized attention for step t is computed as:

$$o_t = \sum_{j=1}^t (\phi(q_t)^\top \phi(k_j)) v_j = \underbrace{\left(\sum_{j=1}^t v_j \phi(k_j)^\top \right)}_{S_t \in \mathbb{R}^{d \times d}} \phi(q_t) \quad (2)$$

By defining $S_t = \sum_{j=1}^t v_j \phi(k_j)^\top$, linear attention essentially implements an element-wise linear recurrence where $S_t = S_{t-1} + v_t \phi(k_t)^\top$ and $o_t = S_t \phi(q_t)$. This formulation achieves linear time complexity $\mathcal{O}(N)$ and requires only constant $\mathcal{O}(1)$ memory per decoding step, effectively eliminating the growing KV-cache.

The Low-Rank Limitation. Despite its computational advantages, linear attention typically underperforms compared to standard softmax attention. Because the output O is computed via an intermediate matrix interaction, its rank is strictly bounded by the feature dimension d , such that $\text{rank}(O) \leq d$. When the number of visual tokens N is significantly larger than d , the attention mechanism is forced to compress the interactions of many tokens into a heavily restricted representational space. This forced compression reduces the model’s ability to capture fine-grained, diverse spatial relationships, hindering its performance on complex vision tasks.

Chunkwise Formulation. To enable sequential importance assignment while maintaining global context, we reformulate linear attention into a causal, chunkwise process. The input sequence of N tokens is partitioned into T non-overlapping chunks of size C . At each step $t \in \{1, \dots, T\}$,

the backbone processes the current chunk into local queries Q_t , keys K_t , and values V_t . Rather than computing a full sequence matrix, we maintain two recurrent memory representations. The numerator state $S_t \in \mathbb{R}^{d \times d}$ accumulates the cross-covariance of keys and values, while the denominator state $Z_t \in \mathbb{R}^{d \times 1}$ tracks the normalization factor:

$$S_t = S_{t-1} + \phi(K_t)^\top V_t \quad (3)$$

$$Z_t = Z_{t-1} + \sum_{i \in \text{chunk } t} \phi(k_{i,t})^\top \quad (4)$$

The attention output for the current chunk t is then computed utilizing these recurrent global states as $O_t = \frac{\phi(Q_t)S_t}{\phi(Q_t)Z_t}$.

3.2. Architecture Overview

We adopt the method from RALA [5] for solving the low-rank problem, and replace the gating component with a multi-agent RL gating framework where every token is treated as an agent that shares a continuous, stochastic Actor-Critic module. We aim to add a budget constraint, and layer-wise information into the gating to improve the linear attention speed and performance. The gating score can be used to prune the token during inference. As show in Figure 1, an input image is tokenized into N patch embeddings and partitioned into T sequential chunks of size C . A linear-attention ViT backbone processes each chunk, projects it into local queries Q_t , keys K_t , and values V_t and maintains a recurrent memory (S_t, Z_t) in place of a full attention matrix. The shared Actor-Critic module evaluates the local token features in parallel. At each layer l , which represents a discrete time step in the Markov Decision Process (MDP), every agent $n \in \{1, \dots, N\}$ produces a scalar gating weight $w_n \in (0, 1)$. For a specific chunk t , the agents collectively produce a weight vector:

$$\mathbf{w}_t = [w_{t,1}, w_{t,2}, \dots, w_{t,C}]^\top \in (0, 1)^C \quad (5)$$

To perform the row-wise memory update, we compute a chunk-level importance factor $\bar{w}_t$ by averaging the actions of the C agents within that specific chunk:

$$\bar{w}_t = \frac{1}{C} \sum_{c=1}^C w_{t,c} \quad (6)$$

These weights are then utilized to gate both the parallel memory update and the recurrent decay:

$$S_t = (1 - \gamma(1 - \bar{w}_t))S_{t-1} + (\mathbf{w}_t \odot \phi(K_t))^\top V_t \quad (7)$$

After all T chunks are processed via the sequential recurrence, the final state (S_T, Z_T) is consumed by the task head to produce the terminal reward R , which is used to guide the cooperative policy optimization.

Training follows a two-phase curriculum. **Phase 1** pre-trains the ViT on the classification task to obtain a visual backbone. Then the RL agents are supervisedly trained with the Fisher Information of the pretrained ViT, avoiding the policy collapse that arises when RL is applied cold to a high-dimensional visual state. **Phase 2** fine-tunes the full system end-to-end on the downstream task via Proximal Policy Optimization (PPO) with Generalized Advantage Estimation. During inference, stochastic sampling is disabled, and the Actor produces deterministic continuous weights with hard pruning per chunk.

3.3. Markov Decision Process Formulation

By reformulating linear attention into a blockwise execution, we isolate the importance evaluation into a continuous MDP. To avoid the autoregressive bottleneck associated with sequential RL evaluation, we isolate the gating policy from the recurrent state update, enabling parallel associative scans across the sequence.

3.3.1. State Space

To enable parallel execution across all T chunks, the agent's observation is localized. For each layer l (representing the temporal step), the state $s_{n,l}$ for agent $n \in \{1, \dots, N\}$ is derived directly from the local visual features:

$$s_{n,l} = \text{Proj}(x_{n,l}), \quad (8)$$

where $x_{n,l}$ is the feature embedding of the n -th token at layer l , and $\text{Proj}(\cdot)$ is a shared lightweight projection.

3.3.2. Action Space

The swarm of N agents utilizes a shared Actor-Critic policy π_θ to evaluate the tokens in parallel. Each agent n outputs a scalar continuous weight $w_{n,l} \in (0, 1)$, representing its relative importance to the task. For a specific chunk t , the agents produce a collective action vector $\mathbf{w}_t$:

$$\mathbf{w}_t = [w_{t,1}, w_{t,2}, \dots, w_{t,C}]^\top \quad (9)$$

3.3.3. Reward Function

To encourage efficient gating, the swarm is governed by a global cooperative reward function calculated at the terminal layer L and broadcasted to the entire swarm. The terminal reward R is defined as:

$$R = \underbrace{-\mathcal{L}_{CE}}_{\text{Task}} - \underbrace{\lambda_b \bar{w}}_{\text{Budget}} - \underbrace{\lambda_s [\bar{w}(1 - \bar{w})]}_{\text{Sparsity}} \quad (10)$$

where $\mathcal{L}_{CE}$ is the standard Cross-Entropy loss for the classification task, $\bar{w}$ is the mean activation across the network, λ_b is a penalty coefficient enforcing the computational budget, and λ_s is a sparsity penalty coefficient. This specific penalty structure actively pushes the agents toward bimodal decisiveness (tending strictly toward 0 or 1) while maintaining overall model performance.

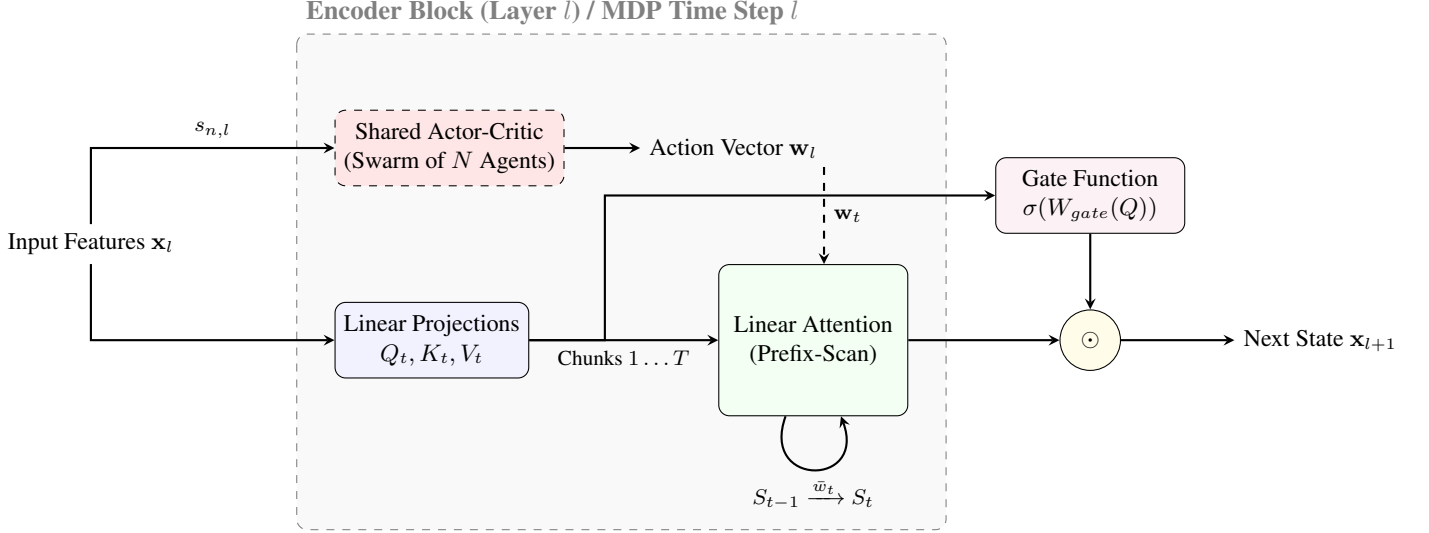


Figure 1. Architecture of the MARL-Gated Linear Attention Block. At each layer l , the swarm of N agents independently evaluates the local features to produce survival weights $\mathbf{w}_l$. These weights dictate the information flow into the chunkwise recurrent memory S_t . Finally, the intermediate linear attention output is refined via a Hadamard-gated transformation to produce the state x_{l+1} for the next layer.

3.3.4. Squashed Gaussian Actor-Critic Network

To enable continuous down-weighting of uninformative features, we model the swarm’s policy $\pi_\theta(a_{n,l} \mid s_{n,l})$ as a **Squashed Gaussian** distribution. This approach avoids the non-differentiability of discrete gating and the competitive constraints of global Softmax operations, allowing each agent to determine its survival weight independently.

The shared Actor network computes the parameters of an unbounded Normal distribution for every agent $n \in \{1, \dots, N\}$ at layer l . To ensure numerical stability, the standard deviation is passed through a softplus activation with a small ϵ floor:

$$\mu_{n,l} = \text{MLP}_\mu(s_{n,l}), \quad \sigma_{n,l} = \text{softplus}(\text{MLP}_\sigma(s_{n,l})) + \epsilon \quad (11)$$

During training, actions are sampled using the reparameterization trick to allow backpropagation through stochastic nodes. The raw sample $x_{n,l}$ is squashed via a hyperbolic tangent and shifted into the valid continuous weighting space of $(0, 1)$:

$$w_{n,l} = \frac{\tanh(x_{n,l}) + 1}{2}, \quad x_{n,l} \sim \mathcal{N}(\mu_{n,l}, \sigma_{n,l}^2) \quad (12)$$

By decoupling the weights from a global sum, the swarm can collectively silence redundant spatial regions (where $w_{n,l} \rightarrow 0$) or amplify high-utility features. However, the non-linear Tanh transformation alters the probability density. To maintain the mathematical integrity of the PPO objective, we apply the **Jacobian correction** for the log-

probability:

$$\log \pi(w_{n,l}) = \log p(x_{n,l}) - \log \left(\frac{1 - \tanh^2(x_{n,l})}{2} + \epsilon \right) \quad (13)$$

Simultaneously, a shared Value network (Critic) $V_\phi(s_{n,l})$ estimates the expected episodic return. This provides a baseline to calculate the advantage $\hat{A}$, reducing variance during the cooperative policy update.

3.4. Pretraining: Information-Saliency Distillation

Initializing a swarm of RL agents from scratch in a high-dimensional visual space often leads to representation collapse, as random initial actions can destroy the backbone’s recurrent state. To circumvent this, Phase 1 initializes the shared Actor-Critic module using empirical **Fisher Information** ($\mathcal{I}_n$) as a saliency oracle.

After warming up the backbone on a dense classification task, we compute the squared gradients of the cross-entropy loss with respect to the token features to approximate their informational utility. These scores, normalized to $\tilde{\mathcal{I}}_n \in [0, 1]$, represent the survival value for each agent. The Actor and Critic are then pretrained via supervised learning to mimic this oracle. The Actor minimizes the Mean Squared Error (MSE) between its deterministic mean prediction and the Fisher utility, while the Critic learns the expected saliency of the state:

$$\mathcal{L}_{\text{Actor}} = \mathbb{E} \left[\sum_{n=1}^N \left(\mu_{n,l}(s_{n,l}) - \tilde{\mathcal{I}}_n \right)^2 \right] \quad (14)$$

$$\mathcal{L}_{\text{Critic}} = \mathbb{E} \left[\sum_{n=1}^N \left(V_{\phi}(s_{n,l}) - \sum_{j=1}^N \tilde{\mathcal{I}}_j \right)^2 \right] \quad (15)$$

where $s_{n,l}$ is the local feature observation of agent n at layer l , and $\tilde{\mathcal{I}}_j$ is the ground-truth Fisher saliency for token j . By forcing every agent’s local Value head to predict the global saliency sum, the swarm develops a shared understanding of the total information available in the image.

3.5. Finetuning: Cooperative Policy Optimization

In Phase 2, the full system is optimized end-to-end via PPO. Using Generalized Advantage Estimation ($\hat{A}$) derived from the Critic, the swarm is updated via the clipped PPO objective. To prevent premature policy convergence, we inject a Jacobian-corrected entropy bonus for the squashed Gaussian distribution:

$$\mathcal{L}_{\text{PPO}} = -\mathbb{E} \left[\min \left(r(\theta) \hat{A}, \text{clip}(r(\theta), 1 \pm \epsilon) \hat{A} \right) \right] - \lambda_{\text{ent}} H(\pi_{\theta}) \quad (16)$$

where the entropy $H(\pi_{\theta})$ accounts for the Tanh-squashing transformation:

$$H(\pi_{\theta}) = \frac{1}{2} \log(2\pi e \sigma^2) + \mathbb{E} \left[\log \left(\frac{1 - \tanh^2(x)}{2} + \epsilon \right) \right] \quad (17)$$

During inference, stochastic sampling is disabled. The shared Actor deterministically produces the weight vector $\mathbf{w}_l \in (0, 1)^N$ for each layer.

4. Experiments

4.1. Datasets

To evaluate the effectiveness of our proposed framework, we conduct experiments on two standard image classification datasets: CIFAR-10 and ImageNet-100. The selection of these specific datasets is driven by a balance between computational feasibility and representative complexity. Training on these subsets significantly reduces the training time per epoch, allowing for rapid iteration and prototyping on accessible consumer-grade hardware. A quantitative comparison of our selected datasets against their full-scale variants is provided in Table 1.

Table 1. Comparison of our selected training datasets against their full-scale variants.

Feature	CIFAR-10	ImageNet-100
Classes	10	100
Resolution	32×32	$\sim 224 \times 224$
Training	50,000	$\sim 130,000$
Validation	10,000	5,000

4.2. Evaluation Metrics

We evaluate the proposed framework against several state-of-the-art models, including architectures based on standard softmax and linear attention mechanisms. Furthermore, we compare our approach against alternative gating methods, such as the sigmoid function. In the current work, our evaluation focuses strictly on the image classification task, measuring Top-1 accuracy. Extending this framework to other dense vision tasks, such as object detection and semantic segmentation, is left for future work.

4.3. Implementation Details

All models are implemented and trained utilizing a combination of a local NVIDIA RTX 5060 Ti and Kaggle Tesla T4 GPU accelerators. Prior to feature extraction, all input images are resized to a fixed spatial resolution of 144×144 . For the MARL reward formulation, the computational budget penalty coefficient (λ_b) is set to 0.05, and the sparsity penalty coefficient (λ_s) is set to 1.0. The entire training process is conducted over 40 epochs without data augmentation, utilizing the Adam optimizer with an initial learning rate of 3×10^{-4} and a learning rate scheduler.

4.4. Results

4.4.1. ImageNet100

Model	Params (M)	Attention Type	Top1-acc (%)
RMT [3]	14	Softmax	57.56
RALVT [5]	14	Linear (Rank-Aug)	69
Baseline (Vanilla)	14	Linear (ELU)	51
Baseline + Sigmoid	14	Linear (Gated)	50.5
Baseline + Softmax	14	Linear (Rank-Aug)	44.24
Baseline + MARL	14	Linear (Gated)	48.88

Table 2. Comparison of models’ accuracy on the ImageNet-100 dataset

In our comparative analysis, we include RMT [3] and RALVT [5] as representative state-of-the-art vision architectures. To specifically isolate the impact of our proposed gating mechanism, we define multiple baseline variants built upon the RALVT framework. The **Vanilla Baseline** consists of a standard linear attention model. The **Baseline + Softmax** variant incorporates the softmax ranking strategy introduced by RALA [5], while the **Baseline + Sigmoid** utilizes a simple sigmoid function for token gating. Finally, **Baseline + MARL** represents our proposed methodology, replacing standard routing methods with our decentralized reinforcement learning gating framework. Our evaluation provided in Table 2 reveals that the proposed token gating mechanism significantly outperforms the softmax-based weighting strategy adopted from RALA. However, the un-gated vanilla baseline still achieves higher accuracy than the

gated variants. We hypothesize that this discrepancy stems from the restricted image resolution (144×144) utilized in our current experiments; at such a small scale, explicitly weighting or gating tokens may inadvertently discard critical spatial features, thereby hindering overall performance. Furthermore, while the RMT and RALVT baselines achieve substantially superior results, this performance gap is primarily attributable to their highly optimized architectures and extensively tuned training recipes.

4.4.2. CIFAR-10

Model	Params (M)	At tention Type	Top1-acc (%)
Baseline (Vanilla)	14	Linear (ELU)	61.92
Baseline + Sigmoid	14	Linear (Gated)	58.77
Baseline + MARL	14	Linear (Gated)	58.37

Table 3. Comparison of models’ accuracy on the CIFAR-10 dataset

Similar trends are observed on the CIFAR-10 dataset as showed in Table 3; however, the performance margins between the models are less pronounced.

4.5. Qualitative Study

Gating Map Visualization To qualitatively evaluate the token selection behavior of our proposed framework against the baselines, we visualize the spatial routing maps generated at deeper layers of the network. Observing the **RALA (Softmax)** map (Figure 2), we see clear visual evidence of the softmax saturation problem discussed earlier; the global normalization across the sequence heavily dilutes the token weights, resulting in a dark, near-zero activation map that chokes information flow. In contrast, the **Sigmoid** (Figure 3) baseline produces an aggressive, high-contrast saliency map that heavily activates on the primary subject while completely zeroing out the background. Our proposed **MARL** (Figure 4) framework produces a softer, continuous gating distribution. We attribute this less decisive gating behavior to current limitations in hyperparameter optimization, suggesting that the sparsity penalty coefficients require further tuning to enforce strict bimodal survival weights.

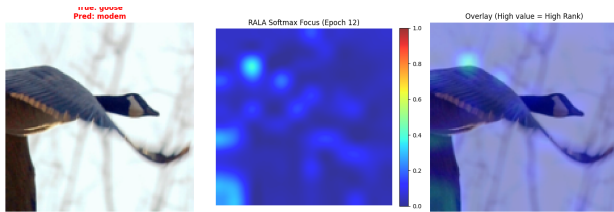


Figure 2. RALA’s saliency map

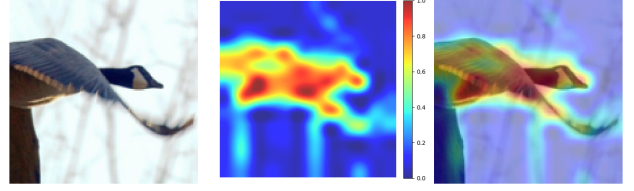


Figure 3. Sigmoid’s saliency map

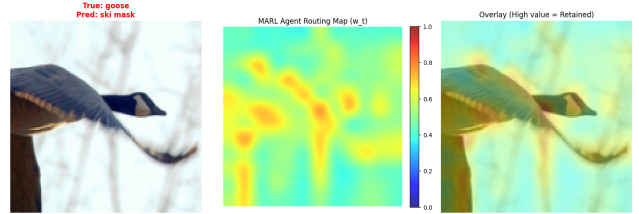


Figure 4. MARL’s saliency map

5. Threats to Validity

In this study, several threats to validity must be acknowledged. The primary assumption of our Phase 1 pretraining is that empirical Fisher Information serves as an accurate proxy for ground-truth visual saliency. However, gradients originating from a global cross-entropy loss may not fully capture complex, multi-token spatial dependencies. Secondly, the inherent optimization variance of PPO poses an internal threat to our findings. Reinforcement learning algorithms are notoriously sensitive to hyperparameter configurations. Specifically, the stability of our model relies heavily on the precise tuning of the budget and sparsity penalty coefficient (λ) and the entropy bonus. Finally, regarding external validity, our current evaluation is strictly constrained to image classification tasks. It remains to be empirically validated how well this localized, independent-agent formulation generalizes to dense prediction tasks, such as high-resolution object detection or semantic segmentation, where preserving precise background context is often crucial. Furthermore, our models were evaluated on a single, fixed image resolution (144×144). This restricted evaluation scope may introduce structural bias, limiting our understanding of the framework’s scalability across diverse visual scales.

6. Conclusion

In this paper, we proposed a MARL gating framework designed to enforce computational budget constraints and aggregate global context, thereby improving the efficiency and representational capability of Linear Attention Vision Transformers. Our model achieved preliminary accuracies of 48.88% and 58.37% on the ImageNet100 and CIFAR-10 datasets, respectively. While the proposed framework remains in its early stages of development, it demonstrates

interpretable saliency maps and yields a measurable accuracy improvement over the existing RALA[5] softmax ranking baseline. Future work will focus on refining the reward optimization process, expanding our empirical evaluation to a broader suite of baseline architectures and higher-resolution configurations, and conducting comprehensive ablation studies to deeply analyze the framework’s internal dynamics.

References

- [1] Han Cai, Junyan Li, Muyan Hu, Chuang Gan, and Song Han. Efficientvit: Multi-scale linear attention for high-resolution dense prediction. In *ICCV*, 2023. [1](#)
- [2] Xiaoyi Dong, Jianmin Bao, Dongdong Chen, Weiming Zhang, Nenghai Yu, Lu Yuan, Dong Chen, and Baining Guo. Cswin transformer: A general vision transformer backbone with cross-shaped windows. In *CVPR*, 2022. [1](#)
- [3] Qihang Fan, Huaibo Huang, Mingrui Chen, Hongmin Liu, and Ran He. Rmt: Retentive networks meet vision transformers. In *CVPR*, 2024. [5](#)
- [4] Qihang Fan, Huaibo Huang, Mingrui Chen, and Ran He. Semantic equitable clustering: A simple and effective strategy for clustering vision tokens. In *ICCV*, 2025. [1](#)
- [5] Qihang Fan, Huaibo Huang, and Ran He. Breaking the low-rank dilemma of linear attention. In *CVPR*, 2025. [1](#), [2](#), [3](#), [5](#), [7](#)
- [6] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024. [2](#)
- [7] Dongchen Han, Xuran Pan, Yizeng Han, Shiji Song, and Gao Huang. Flatten transformer: Vision transformer using focused linear attention. In *ICCV*, 2023. [1](#), [2](#)
- [8] Dongchen Han, Ziyi Wang, Zhuofan Xia, Yizeng Han, Yifan Pu, Chunjiang Ge, Jun Song, Shiji Song, Bo Zheng, and Gao Huang. Demystify mamba in vision: A linear attention perspective. In *NeurIPS*, 2024. [2](#)
- [9] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *ICML*, 2020. [1](#)
- [10] Kai Liu, Tianyi Wu, Cong Liu, and Guodong Guo. Dynamic group transformer: A general vision transformer backbone with dynamic group attention. *IJCAI*, 2022. [1](#)
- [11] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021. [1](#)
- [12] Chenglong Lu, Shen Liang, Xuwei Wang, and Wei Wang. Reinforcement learning-based token pruning in vision transformers: A markov game approach. In *ICME*, 2025. [2](#)
- [13] Jiachen Lu, Junge Zhang, Xiatian Zhu, Jianfeng Feng, Tao Xiang, and Li Zhang. Softmax-free linear transformers. In *IJCV*, 2024. [1](#)
- [14] Lingchen Meng, Hengduo Li, Bor-Chun Chen, Shiyi Lan, Zuxuan Wu, Yu-Gang Jiang, and Ser-Nam Lim. Adavit: Adaptive vision transformers for efficient image recognition. In *CVPR*, 2022. [2](#)
- [15] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. Dynamicvit: Efficient vision transformers with dynamic token sparsification. In *NeurIPS*, 2021. [1](#), [2](#)
- [16] Michael S Ryoo, AJ Piergiovanni, Anurag Arnab, Mostafa Dehghani, and Anelia Angelova. Tokenlearner: What can 8 learned tokens do for images and videos? *arXiv preprint arXiv:2106.11297*, 2021. [2](#)
- [17] Zhuoran Shen, Mingyuan Zhang, Haiyu Zhao, Shuai Yi, and Hongsheng Li. Efficient attention: Attention with linear complexities. In *WACV*, 2021. [1](#), [2](#)
- [18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017. [1](#)